MARK SCHEME · TEACHER COPY

10BR · Chapter 8: Programming

Cambridge IGCSE Computer Science 0478

Paper 2 Algorithms, Programming and Logic

Total marks: 75

Official Cambridge mark scheme extracts for each question in the matching exam paper, with question numbers changed to match the exam. Source of each question is shown above its rows.

For teacher / admin use only. Do not give to students before they have attempted the paper.

Mark scheme extracts © Cambridge University Press & Assessment.

Total: 75 marks

Question 1 · May/June 2023 Paper 22, original Q6 [6 marks]

Question	Answer	Marks
1	**One mark** for each feature and **one mark** for corresponding example (**max six**) • ensuring that all identifiers have meaningful names … • … example using `Total` to store a running total • using comments to explain how the program works … • … example `// all values are zeroed before the next calculation` • using procedures and functions for the tasks within a program … • … example `CalculateInterest(Deposit, Rate)`	6

Question 2 · Oct/Nov 2023 Paper 23, original Q10 [15 marks]

Question	Answer	Marks

2	• AO2 (maximum 9 marks)
	• AO3 (maximum 6 marks)
	
	Data Structures required names shown underlined must be used as given in the scenario
	2D Array or list `Temperatures`
	Variables `MaxDay`, `MinDay`, `AvDay`, `MaxWeek`, `MinWeek`, `AvWeek`
	
	Requirements (techniques)
	
	R1 Find maximum and minimum temperatures for each day and calculates the average daily temperature (searching, totalling)
	R2 Find maximum and minimum temperatures for week and calculates the average weekly temperature (nested searching, totalling)
	R3 outputs for each day name, the rounded values for maximum temperature, minimum temperatures and average temperature. Outputs for the week the rounded values for maximum temperature, minimum temperatures and average temperature (output with appropriate messages and rounded values)
	
	Example 15-mark answer in pseudocode:
	
	`// meaningful identifier names and appropriate data structures to store the data required`
	`DECLARE DayCounter, HourCounter : INTEGER`
	`DECLARE AvDay, AvWeek, MaxDay, MinDay, MaxWeek, MinWeek : REAL`
	`DECLARE DayTotal, WeekTotal : REAL`
	`DECLARE Day : STRING`
	
	`CONSTANT Hours ← 24`
	`CONSTANT Days ← 7`
		15

Total: 75 marks

2

```
MaxWeek ← -1000// initialise max and min temperatures and total for the week
MinWeek ← 1000
WeekTotal ← 0

FOR DayCounter ← 0 TO Days - 1
    MaxDay ← -1000// initialise max and min temperatures and total for each day
    MinDay ← 1000
    DayTotal ← 0
    FOR HourCounter ← 0 TO Hours - 1
        DayTotal ← DayTotal + Temperatures(HourCounter, DayCounter)
          // update total maximum and minimum
        IF Temperatures(HourCounter, DayCounter) > MaxDay
          THEN
            MaxDay ← Temperatures(HourCounter, DayCounter)
        ENDIF
        IF Temperatures(HourCounter, DayCounter) < MinDay
          THEN
            MinDay ← Temperatures(HourCounter, DayCounter)
        ENDIF
    NEXT HourCounter

    CASE OF DayCounter  // select message for day
      0 : Day ← "Monday"
      1 : Day ← "Tuesday"
      2 : Day ← "Wednesday"
      3 : Day ← "Thursday"
      4 : Day ← "Friday"
      5 : Day ← "Saturday"
      6 : Day ← "Sunday"

    ENDCASE

    DayAverage ← DayTotal / Hours  // output results for day
    OUTPUT Day  // Results from a day
    OUTPUT "Maximum temperature ", MaxDay
    OUTPUT "Minimum temperature ", MinDay
    OUTPUT "Average temperature ", ROUND(DayAverage,2)
```

Total: 75 marks

2	IF MaxDay > MaxWeek // update total maximum and minimum THEN MaxWeek ← MaxDay ENDIF IF MinDay > MinWeek THEN MinWeek ← MinDay ENDIF WeekTotal ← WeekTotal + DayTotal // update total for week NEXT DayCounter WeekAverage ← WeekTotal / Days OUTPUT "Maximum temperature for week ", MaxWeek// output results for week OUTPUT "Minimum temperature for week ", MinWeek OUTPUT "Average temperature for Week ", ROUND(WeekAverage,2)	

Marking Instructions in italics
Marking Instructions in italics

Question 3 · Feb/March 2024 Paper 22, original Q6 [4 marks]

Question	Answer	Marks
3	**One** mark for identifying a type of iteration, **one** mark for accompanying description **max four** • count controlled (1) number of iterations is pre-determined (1) • pre-condition (1) checks condition at start of loop // loop may not iterate (1) • post-condition (1) checks condition at end of loop // loop always iterates at least once (1)	**4**

Total: 75 marks

Question 4 · May/June 2024 Paper 22, original Q11 [15 marks]

Question	Answer	Marks

4	**Data Structures required** with names as given in the scenario:
	Arrays or lists `Grid`
	
	Requirements (techniques)
	R1 Set up game – generate random cell, clear all other cells in array, set player start position and start player moves counter (iteration, use of arrays and library routines (round and random))
	R2 Input and check move – is it valid? (input, output, iteration and selection)
	R3 Decide outcome – has move found the X? If so, give appropriate output. If not increment counter and continue. If 10 moves exceeded, give appropriate output (use of arrays, iteration, selection and output).
	
	Example 15-mark answer in pseudocode
	`// Set up game`
	`FOR Row ← 1 TO 5`
	`  FOR Column ← 1 TO 5`
	`  Grid[Row, Column] ← ''// set grid cells to be empty`
	`  NEXT Column`
	`NEXT Row`
		15

4

```
REPEAT  // not in cell 1,1
    XRow ← ROUND ((RANDOM() * 4) + 1, 0) // Random row position between 1 and 5 in GRID
    XColumn ← ROUND ((RANDOM() * 4) + 1, 0) // Random column position between 1 and 5 in
GRID
UNTIL XRow <> 1 and XColumn <> 1 // not in cell 1,1

Grid [XRow, XColumn] ← 'X'
MaxMove ← 10
NumberMoves ← 0
PlayerRow ← 1
PlayerColumn ← 1
Win ← FALSE

// during game
WHILE NumberMoves < MaxMove AND NOT Win
    MoveError ← FALSE
    OUTPUT "Please enter your move, L - Left, R - Right, U - Up or D - Down"
    INPUT UPPER(PlayerMove)
    REPEAT
        CASE OF PlayerMove
          'L' : TempColumn ← PlayerColumn - 1
          'R' : TempColumn ← PlayerColumn + 1
          'U' : TempRow ← PlayerRow - 1
          'D' : TempRow ← PlayerRow + 1
          OTHERWISE MoveError ← TRUE
        ENDCASE

// check for out-of-range moves
        IF TempColumn < 1 or TempColumn > 5
          THEN
            MoveError ← TRUE
          ELSE
            PlayerColumn ← TempColumn
        ENDIF
```

Total: 75 marks

4	IF TempRow < 1 or TempRow > 5 THEN MoveError ← TRUE ELSE PlayerRow ← TempRow ENDIF // check win if X Found IF Grid [PlayerRow, PlayerColumn] = 'X' THEN OUTPUT "You Win" Win ← TRUE ELSE IF NOT MoveError THEN NumberMoves ← NumberMoves + 1 ENDIF ENDIF UNTIL NOT MoveError ENDWHILE IF NOT Win THEN OUTPUT "You Lose" ENDIF	

Marking Instructions in italics
AO2: Apply knowledge and understanding of the principles and concepts of computer science to a given context, including the analysis and design of computational or programming problems

Question 5 · Oct/Nov 2024 Paper 21, original Q6 (a),(b),(c) [9 marks]

Question	Answer	Marks
5(a)	**One** mark for each point (**max three**) • 06 T ← 0 • 08 IF A[C] = 0 • 14 NEXT C // NEXT	3

5(b)	**One** mark for outputting `T`
	One mark for outputting `I`
	One mark for suitable messages
	
	Example:
	`15 OUTPUT "Number of zeros in the array is ", T`
	`16 OUTPUT "Total of non-zero elements in the array is ", I`
		3

5(c)	**One** mark for a meaningful identifier for the array
	`A MyArray`
	
	One mark for 2 meaningful identifiers, **Two** marks for 3 meaningful identifiers:
	`T Counter`
	`C Index`
	`I Total`
		3

Question 6 · May/June 2025 Paper 22, original Q5 (a),(b),(c) [11 marks]

Question	Answer	Marks

6(a)	**One** mark per mark point • Line 01 / `DECLARE Names : ARRAY[1:500] OF REAL` should be `DECLARE Names : ARRAY[1:500] OF STRING` • Line 09 / Total ← `100` should be Total ← `0` • Line 11 / `INPUT Names[Index]` should be `INPUT Names[Counter]` • Line 13 / `Total ← Counter + Heights[Counter]` should be `Total ← Total + Heights[Counter]` • Line 20 / `OUTPUT "The shortest height is ", Heights` should be `OUTPUT "The shortest height is ", Shortest` or `OUTPUT "The shortest height is ", Heights[Index]`	**5**

Scan to open the iLearnCS site

Total: 75 marks

```
01 DECLARE Names : ARRAY[1:500] OF STRING
02 DECLARE Heights : ARRAY[1:500] OF REAL
03 DECLARE Shortest : REAL
04 DECLARE Total : REAL
05 DECLARE Counter : INTEGER
06 DECLARE Index : INTEGER
07 Shortest ← 500
08 Index ← 0
09 Total ← 0
10 FOR Counter ← 1 TO 500
11     INPUT Names[Counter]
12     INPUT Heights[Counter]
13     Total ← Total + Heights[Counter]
14     IF Heights[Counter] < Shortest
15       THEN
16         Shortest ← Heights[Counter]
17         Index ← Counter
18     ENDIF
19 NEXT Counter
20 OUTPUT "The shortest height is ", Shortest
21 OUTPUT "The shortest person is ", Names[Index]
22 OUTPUT "The average height is ", Total / 500
```

6(b)	**One** mark per mark point MP1 Output statement with correct calculation of average, or use of a variable average. MP2 Correct use of `ROUND` function set to 1 decimal place. **For example,** `OUTPUT ROUND(Total / 500, 1)`	**2**
6(c)	**One** mark per statement, **max four** MP1 Declaration of new variable for tallest person / largest value at start of algorithm MP2 Initialisation of largest variable to a low number e.g. 0 MP3 New selection statement after input of height to compare it with current largest value MP4 If input of height is larger than current largest variable, it should become the new largest value MP5 **Outside the loop**, output the current value of the largest variable.	**4**

Total: 75 marks

Question 7 · Oct/Nov 2025 Paper 21, original Q11 [15 marks]

Question	Answer	Marks
7	Use the tables for A02 and A03 below to award a mark in a suitable band using a best fit approach Then add up the total. Marks are available for: • AO2 (maximum 9 marks) • AO3 (maximum 6 marks) **Data Structures required** names shown underlined must be used as given in the scenario Arrays or lists: `CompetitorName`, `CompetitorScore` **Requirements (techniques)** R1 inputs the score for each round validating the input is in the range 0 to 30 inclusive (input, output, nested iteration, validation) R2 calculates the score of each competitor discarding the highest and lowest scores of the 10 rounds (nested iteration, output and selection) R3 outputs the name of the competitor and whether they have qualified, are a reserve or not qualified, output the number of competitors who have qualified, are reserve and not qualified (totalling, output and selection)	**15**

7

Example 15-mark answer in pseudocode.

```
// programming techniques of iteration, selection, counting, totalling and output are used
// Candidates did not need to declare any arrays or variables.
DECLARE Score : INTEGER
DECLARE Highest : INTEGER
DECLARE Lowest : INTEGER
DECLARE Qualified : INTEGER
DECLARE Reserve : INTEGER
DECLARE NotQualified : INTEGER

Qualified ← 0
Reserve ← 0
NotQualified ← 0

FOR X ← 1 TO 30  //looping for number of competitors
    FOR Round ← 1 TO 10  //looping for number of rounds
        OUTPUT CompetitorName[X]
        OUTPUT "Enter the score for the round: ", Round
        INPUT Score
        // validation of input for between 0 and 30 inclusive
        WHILE Score < 0 OR Score > 30 DO
            OUTPUT "The score must be between 0 and 30 inclusive. Please try again."
            INPUT Score
        ENDWHILE
        CompetitorScore[X,  Round] ← Score  // adding score to 2-D array
    NEXT Round
NEXT X


// or suitable method of determining how many competitors
FOR X ← 1 TO 30
    Count ← 0
    Highest ← 0
    Score ← 0
    Lowest ← 999 // resetting variables for each competitor
```

7	FOR Y ← 1 TO 10 // loop through rounds Score ← Score + CompetitorScore[X, Y] IF CompetitorScore[X, Y] > Highest // finds highest score THEN Highest ← CompetitorScore[X, Y] ENDIF IF CompetitorScore[X, Y] < Lowest // finds lowest score THEN Lowest ← CompetitorScore[X, Y] ENDIF NEXT Y Score ← Score - Highest - Lowest // calculating score to subtract highest and lowest IF Score >= 210 // competitor has qualified THEN OUTPUT "Competitor ", CompetitorName[X], " has qualified" Qualified ← Qualified + 1 ELSE IF Score >= 180 AND Score <= 209 // competitor is a reserve competitor THEN OUTPUT "Competitor ", CompetitorName[X], " are a reserve" Reserve ← Reserve + 1 ELSE OUTPUT "Competitor ", CompetitorName[X], " has not qualified" // competitor not qualified NotQualified ← NOTQualified + 1 ENDIF NEXT X OUTPUT "The number of competitors qualified is ", Qualified // output of results OUTPUT "The number of competitors who are a reserve is ", Reserve OUTPUT "The number of competitors not qualified is ", NotQualified	
7 1	**Marking Instructions in italics**	

AO2: Apply knowledge and understanding of the principles and concepts of computer science to a given context, including the analysis and design of computational or programming problems			
0	**1–3**	**4–6**	**7–9**
No creditable response.	At least one programming technique has been used. *Any use of selection, iteration, counting, totalling, input and output.*	Some programming techniques used are appropriate to the problem. *More than one technique seen applied to the scenario, check the list of techniques needed.*	The range of programming techniques used is appropriate to the problem. *All criteria stated for the scenario have been covered by the use of appropriate programming techniques, check the list of techniques needed.*
	Some data has been stored but not appropriately. *Any **use** of variables or arrays or other language dependent data structures e.g. Python lists.*	Some of the data structures chosen are appropriate and store some of the data required. *More than one data structure **used** to store data required by the scenario.*	The data structures chosen are appropriate and store all the data required. *The data structures **used** store all the data required by the scenario.*

7

Marking Instructions in italics

Scan to open the iLearnCS site

AO3: Provide solutions to problems by:
- **evaluating computer systems**
- **making reasoned judgements**
- **presenting conclusions**

0	1–2	3–4	5–6
No creditable response.	Program seen without relevant comments.	Program seen with some relevant comment(s).	The program has been fully commented.
	Some identifier names used are appropriate. *Some of the data structures used have meaningful names.*	The majority of identifiers used are appropriately named. *Most of the data structures used have meaningful names.*	Suitable identifiers with names meaningful to their purpose have been used throughout. *All of the data structures used have meaningful names.*
	The solution is illogical.	The solution contains parts that may be illogical.	The program is in a logical order.
	The solution is inaccurate in many places. *Solution contains few lines of code with errors that attempt to perform a task given in the scenario.*	The solution contains parts that are inaccurate. *Solution contains lines of code with some errors that logically perform tasks given in the scenario. Ignore minor syntax errors.*	The solution is accurate. *Solution logically performs all the tasks given in the scenario. Ignore minor syntax errors.*
	The solution attempts at least one of the requirements. *Solution contains lines of code that attempt at least one task given in the scenario.*	The solution meets most of the requirements. *Solution contains lines of code that attempts most tasks given in the scenario.*	The solution meets all the requirements given in the question. *Solution performs all the tasks given in the scenario.*